



PARALLEL PROGRAMMING PROJECT

Matchmaking Service

A 5v5 matchmaking system built with `java.util.concurrent`

What is it?

A small 5v5 matchmaking service inspired by competitive games.

- Players join a queue with an MMR (skill rating).
- A background matchmaker continuously pairs 10 skill-close players into two balanced teams.
- Bots are constantly added to the queue, so the service is always alive.
- Same kind of problem solved by CS2, Valorant, LoL and Dota 2 — at student scale.

Tech stack

BACKEND

Java 17+

- `com.sun.net.httpserver.HttpServer` (built-in JDK)
- `java.util.concurrent` (all parallelism)
- Jackson (JSON serialization)
- Maven (build)

FRONTEND

React 18 + Vite

- Plain JavaScript (no TypeScript)
- 4 components (Login, Lobby, Queue, Match)
- LiveStats sidebar (always visible)
- Polls `/api/stats` every 1 s

No frameworks. No databases. Just the JDK + React. Everything lives in memory while the server runs.

Four parallel components

1 HTTP request pool

8 worker threads

Serves the REST API. Multiple browsers can register, queue and poll at the same time.

2 Matchmaker tick

1 thread, every 500 ms

Drains the queue, sorts by MMR, finds 10 skill-close players, builds two balanced teams.

3 Bot populator

1 thread, every 500 ms

Keeps the queue full of bots so the service is always alive. Target scales with worker count.

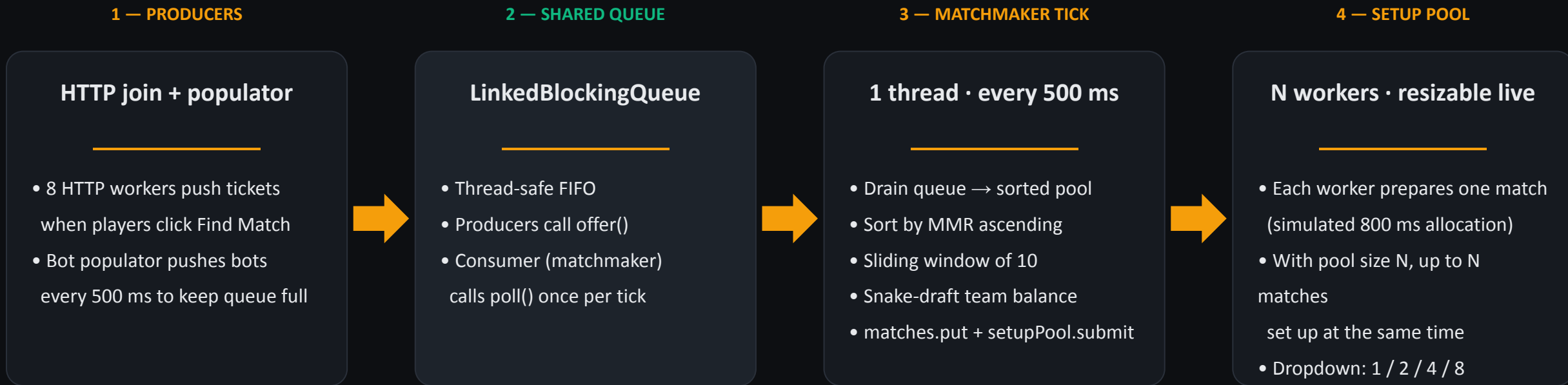
4 Match-setup pool

N workers (configurable)

After a match is formed, setup work runs here. More workers $\Rightarrow$ more matches prepared in parallel.

How a player flows through the thread pools

Four threads, two shared structures. Every horizontal arrow crosses a thread boundary.



PARALLEL READ PATH

```
GET /api/queue/status → ConcurrentHashMap.get(playerId) → match payload (no lock; all polls in parallel)
```

java.util.concurrent primitives used

<code>LinkedBlockingQueue<QueueTicket></code>	Producer/consumer hand-off — HTTP threads push, matchmaker drains.
<code>ScheduledExecutorService × 2</code>	Matchmaker tick and bot populator on separate threads.
<code>ThreadPoolExecutor × 2</code>	HTTP request pool and match-setup pool (resizable live).
<code>ConcurrentHashMap</code>	Lock-free reads of player & match registries under heavy polling.
<code>ConcurrentLinkedQueue / AtomicLong</code>	Lock-free counters: tick count, match IDs, bots spawned.
<code>ReentrantLock</code>	Critical section around the sorted waiting pool inside one tick.
<code>volatile + daemon threads</code>	Cross-thread visibility for last-tick duration; clean shutdown.

How the matchmaker forms a match

Every 500 ms, on a dedicated background thread:

- 1. DRAIN** Pull new arrivals from the BlockingQueue into a sorted pool.
- 2. FILTER** Drop tickets for players who cancelled.
- 3. SORT** Sort the pool by MMR ascending — adjacent players are skill-close.
- 4. SCAN** Slide a window of 10. If the MMR spread fits the tolerance, form a match.
- 5. RELAX** Tolerance starts at 300 MMR and widens 100 MMR / second waited (cap 1500).
- 6. BALANCE** Sort the 10 by MMR desc, assign A,B,B,A,A,B,B,A,A,B — snake draft.
- 7. HAND OFF** Submit `setupMatch()` to the parallel setup pool. Tick continues without blocking.

Snake draft keeps teams fair

Sort 10 players by MMR (highest first), then assign in pattern A B B A A B B A A B.

TEAM A

8450 MMR

Player #1	2000 MMR
Player #4	1800 MMR
Player #5	1700 MMR
Player #8	1500 MMR
Player #9	1450 MMR

TEAM B

8400 MMR

Player #2	1900 MMR
Player #3	1850 MMR
Player #6	1650 MMR
Player #7	1600 MMR
Player #10	1400 MMR

Total difference: 50 MMR. Typical match: under 100 MMR across two 5-stacks.

Live parallelism, visible in the UI

An always-visible sidebar shows the four threads working in real time.

- **Two big counters**
Players in queue and total matches formed.
- **Pulse indicators**
Green dot lights up when a thread is busy.
- **Capacity bars**
Visualize how many setup workers are running right now.
- **Live dropdown**
Resize the setup pool live: 1, 2, 4 or 8 workers.
- **Visible speedup**
Switch from 1 to 8 workers — match throughput jumps ~6×.

LIVE SYSTEM ACTIVITY

78

in queue

42

matches formed

- **Matchmaker tick**
tick #142 · 3 ms
- **Bot populator**
264 bots spawned
- **Match-setup pool**
4 / 4 workers busy
- **HTTP request pool**
1 / 8 workers busy

How to run the demo

1. Start the backend (in one terminal)

```
cd backend && mvn compile exec:java
```

2. Start the frontend (in another terminal)

```
cd frontend && npm install && npm run dev
```

3. Open the app and play

- Open <http://localhost:5173> — enter any name to register.
- Click 'Find Match' — you'll be matched within a second (the queue is full of bots).
- Try the 'Match-setup workers' dropdown: switch from 1 to 8 and watch the capacity bar fill up.

What this project demonstrates

1

Real-world domain

Game matchmaking — the same class of problem CS2, Valorant and LoL solve.

2

Four cooperating thread pools

HTTP workers, matchmaker tick, bot populator, match-setup pool.

3

Seven concurrency primitives

BlockingQueue, ScheduledExecutorService, ThreadPoolExecutor, ConcurrentHashMap, AtomicLong, ReentrantLock, volatile.

4

Live visualization

The UI shows every thread pool busy in real time.

5

Tunable parallelism

Resize the setup pool from 1 to 8 workers live — visible speedup.